

# Supplementary Material: A Differentiable Atari VCS: A Complex, Fully Known Ground Truth for Explainable AI

Anonymous submission

This supplement gives the full proofs of the two theorems stated in the main paper, together with the implementation-effort plot referenced in the Results. It is self-contained: we first restate the soft/hard formulation, then prove the forward-equivalence and temperature-limit results, and finally show the effort timeline.

## 1 Setup and Notation

The differentiable emulator runs one of two transition maps over the full VCS state  $x = (\text{registers}, \text{RAM}, \text{RIOT}, \text{TIA}, \text{frame buffer})$ : the bit-exact *hard* map  $\Phi_H$  and the differentiable *soft* map  $\Phi_S$ . Both are the per-opcode handler selected by the byte at the program counter (pc). Four primitives appear in the handlers.

*One-hot reads.* A ROM or RAM read at integer address  $a$  is the dot product

$$\text{peek}(r, a) = \mathbf{1}_a^\top r = \sum_i \llbracket i = a \rrbracket r_i = r_a. \quad (1)$$

*Softmax dispatch.* With a score vector (logits)  $\ell \in \mathbb{R}^K$  over the  $K = 256$  opcodes, temperature  $T > 0$ , and softmax weights  $w = \text{softmax}(\ell/T)$ , the relaxed dispatch output over candidate handler rows  $V$  is

$$\text{select}(\ell, V; T) = w^\top V, \quad w_k = \frac{e^{\ell_k/T}}{\sum_j e^{\ell_j/T}}. \quad (2)$$

*Sigmoid branch.* For a relaxed status flag  $f \in [0, 1]$  written as a logit  $z = s(2f - 1)$  (sign  $s$  for branch-when-set/clear) and sharpness  $\alpha$ , the gate and relaxed program counter are

$$g = \sigma(\alpha z), \quad \text{pc}_{\text{soft}} = (1 - g) \text{pc}_{\bar{b}} + g \text{pc}_b. \quad (3)$$

*Straight-through estimator.* For a soft value with a matching hard value,

$$\text{STE}(\text{soft}, \text{hard}) = \text{soft} + \text{sg}(\text{hard} - \text{soft}), \quad (4)$$

whose forward value is *hard* and whose gradient is that of soft.

In the executed soft step, dispatch uses the saturated (hard) form of (2), and the branch returns  $\text{STE}(\text{pc}_{\text{soft}}, \text{pc}_{\text{hard}})$ ; (2) and (3) in their relaxed form define the “fully relaxed” variant analysed in Theorem 2.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

**Assumption (DM, decision margins).** Along the executed trajectory of length  $N$ , every discrete decision has a strictly positive margin: each dispatch has a logit gap  $\Delta = \ell_{k^*} - \max_{k \neq k^*} \ell_k \geq \Delta_{\min} > 0$ , and each branch has a margin  $m = |z| \geq m_{\min} > 0$ .

## 2 Proofs

**Theorem 1 (exact forward equivalence).** *For every reachable state  $x$  and every finite  $\alpha, T$ ,  $\Phi_S(x) = \Phi_H(x)$ ; hence  $x_t^S = x_t^H$  for all  $t \leq N$ . The two modes differ only in the Jacobian.*

*Proof.* It suffices to show that, for each opcode, the soft and hard handlers produce the same output value; the state is then carried forward identically. Each handler is composed of four kinds of primitive.

(i) *Memory reads.* A ROM or RAM read at integer address  $a$  is the one-hot dot product (1). Because  $a$  is an integer index,  $\mathbf{1}_a$  is exact and  $\text{peek}(r, a) = r_a$ , identical to a hard array read.

(ii) *Arithmetic.* Register, status-flag, binary-coded-decimal (BCD), and RIOT-timer updates use the same integer and round operations as the hard handlers; the soft path stores these quantities as float32 but computes them with integer semantics, so their outputs are equal.

(iii) *Dispatch.* The handler is chosen by a hard switch on the decoded opcode byte, exactly as in hard mode; the softmax form (2) is not used on the forward path.

(iv) *Branch.* A conditional branch forms  $\text{pc}_{\text{soft}}$  from (3) but returns  $\text{STE}(\text{pc}_{\text{soft}}, \text{pc}_{\text{hard}})$  (4). As  $\text{sg}$  is the identity on the forward pass, this equals  $\text{pc}_{\text{hard}}$  for every  $\alpha$  and  $T$ .

Each primitive’s forward output thus equals the hard handler’s, so  $\Phi_S(x) = \Phi_H(x)$  for every reachable  $x$ ; the hard handlers are themselves validated bit-for-bit against XITARI by the conformance harnesses. By induction over the  $N$  steps of a trajectory,  $x_t^S = x_t^H$  for all  $t \leq N$ .

*Bit-exactness.* float32 represents every integer in  $[0, 2^{24}]$  exactly, and all VCS quantities lie in this range—data bytes are at most 255, addresses are 13 bits, and per-frame cycle counts are a few tens of thousands—so the float32 results are not merely close to the integer results but identical.

*Gradients.* The stop-gradient in (iv) alters only the reverse-mode rule: it sets  $\partial \text{STE} = \partial \text{pc}_{\text{soft}}$ , the derivative of the sigmoid-blended counter, which is generically nonzero where the hard branch (a step function of the flag) has zero

or undefined derivative; likewise the read in (i) has Jacobian  $1_a$  with respect to  $r$ . The forward pass is unchanged while gradients become available.  $\square$

**Theorem 2 (temperature-limit bound).** *For the fully relaxed variant (softmax dispatch and sigmoid branch, no straight-through correction), under Assumption (DM),*

$\|\Phi_S^T(x) - \Phi_H(x)\|_\infty \leq C[(K-1)e^{-\Delta_{\min}/T} + e^{-\alpha m_{\min}}]$ ,  
and over  $N$  steps with Lipschitz constant  $L \geq 1$  the trajectory error is at most  $\frac{L^N - 1}{L - 1}$  times this, which is  $\mathcal{O}(e^{-c/T})$  with  $c = \Delta_{\min}$ .

*Proof.* We bound the per-step deviation, then propagate it. Throughout,  $C$  bounds the range of any state value (bytes  $\leq 255$ , branch offsets  $\leq 256$ ).

*Dispatch term.* At a step with logits  $\ell$  and winner  $k^*$ , the gap is  $\Delta \geq \Delta_{\min} > 0$  by Assumption (DM). For any loser  $k \neq k^*$ ,

$$w_k = \frac{e^{\ell_k/T}}{\sum_j e^{\ell_j/T}} \leq \frac{e^{\ell_k/T}}{e^{\ell_{k^*}/T}} = e^{(\ell_k - \ell_{k^*})/T} \leq e^{-\Delta_{\min}/T},$$

so the non-winning mass is  $1 - w_{k^*} = \sum_{k \neq k^*} w_k \leq (K-1)e^{-\Delta_{\min}/T}$ . As the output is the convex combination  $\sum_k w_k V_k$ ,

$$\begin{aligned} \left\| \sum_k w_k V_k - V_{k^*} \right\|_\infty &\leq (1 - w_{k^*}) \max_k \|V_k - V_{k^*}\|_\infty \\ &\leq (K-1)e^{-\Delta_{\min}/T} C. \end{aligned}$$

*Branch term.* For a branch with margin  $m = |z| \geq m_{\min}$ , the gate error is  $|\sigma(\alpha z) - \mathbb{I}[z > 0]| = \sigma(-\alpha m) \leq e^{-\alpha m_{\min}}$ , so the blended counter  $(1-g)\text{pc}_{\bar{b}} + g\text{pc}_b$  differs from the hard target by at most  $|\text{pc}_b - \text{pc}_{\bar{b}}|e^{-\alpha m_{\min}} \leq C e^{-\alpha m_{\min}}$ .

*Per-step bound.* Adding the two contributions gives, for every reachable  $x$ ,  $\delta := \|\Phi_S^T(x) - \Phi_H(x)\|_\infty \leq C[(K-1)e^{-\Delta_{\min}/T} + e^{-\alpha m_{\min}}]$ .

*Propagation.* Let  $L \geq 1$  be a Lipschitz constant of  $\Phi_H$  in  $\|\cdot\|_\infty$  and  $e_t = \|x_t^{T,S} - x_t^H\|_\infty$  the trajectory error after  $t$  steps, with  $e_0 = 0$ . Each step adds a deviation of at most  $\delta$  to the hard map applied to the accumulated error, giving  $e_{t+1} \leq L e_t + \delta$ ; unrolling,  $e_N \leq \delta \sum_{i=0}^{N-1} L^i = \delta (L^N - 1)/(L - 1)$ . As  $T \rightarrow 0$  (and  $\alpha \rightarrow \infty$ , since  $m_{\min} = \Theta(1)$ ),  $\delta = \mathcal{O}(e^{-\Delta_{\min}/T}) \rightarrow 0$ , so the relaxed trajectory converges to the hard one and  $\Phi_S^T \rightarrow \Phi_H$ .  $\square$

### 3 Effect of the Relaxation Parameters on the Gradient

Theorems 1 and 2 say the hard limit ( $\alpha \rightarrow \infty$  for the sigmoid branch,  $T \rightarrow 0$  for the softmax select) is exact in value but degenerate in gradient. Figure 1 shows this directly on jutari, as the relaxation values are swept. We render a single player cannon whose horizontal position is produced by the soft primitives—a sigmoid-gated blend of a left/right placement (`soft_branch`, sharpness  $\alpha$ ) in the top row, and a softmax mixture over candidate columns (`soft_select`, temperature  $T$ ) in the bottom row—and plot the screen-space directional-derivative saliency  $\partial(\text{screen})/\partial(\text{control})$

$\partial(\text{screen})/\partial(\text{control})$  vs. relaxation strength (vanishes in the hard limit  $\alpha \rightarrow \infty$ ,  $T \rightarrow 0$ )

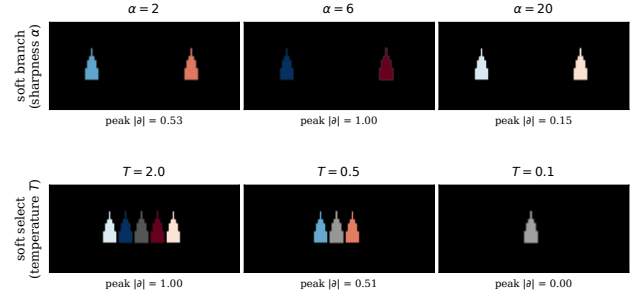


Figure 1: Screen-space gradient  $\partial(\text{screen})/\partial(\text{control})$  on jutari as the relaxation strength is swept. **Top:** sigmoid branch (`soft_branch`), increasing sharpness  $\alpha$ . **Bottom:** softmax select (`soft_select`), decreasing temperature  $T$ . Blue/red are the two motion directions; the per-panel number is the peak gradient magnitude relative to the row maximum. The gradient is largest at intermediate relaxation and vanishes toward the hard limit (large  $\alpha$ , small  $T$ ), consistent with Theorems 1 and 2.

(blue/red = the two motion directions), with a shared colour scale per row.

The trend matches the theory. For the branch, the gradient is strongest at a moderate  $\alpha$  and collapses as  $\alpha$  grows ( $\alpha=2, 6, 20$  give peak magnitudes 0.53, 1.00, 0.15): a very sharp gate is saturated, so its derivative  $\alpha \sigma(\alpha z)(1 - \sigma(\alpha z))$  is near zero away from the decision. For the select, lower  $T$  concentrates the mixture (peak magnitudes 1.00, 0.51, 0.00 for  $T=2.0, 0.5, 0.1$ ): at  $T=0.1$  the pick is effectively one-hot, so a small change in the control no longer moves the chosen column and the gradient vanishes between candidate boundaries. In both cases the forward render stays exact (Theorem 1); only the gradient changes, and it is most informative at intermediate relaxation. This is a qualitative study of how the parameter *values* act on the gradient, run on jutari; we did not need to tune them for the bit-exact forward pass.

### 4 Implementation Effort

Figure 2 visualises the effort estimate reported in the main paper: 373 commits over a 29-day calendar span, of which the active sessions sum to about 137 hours.

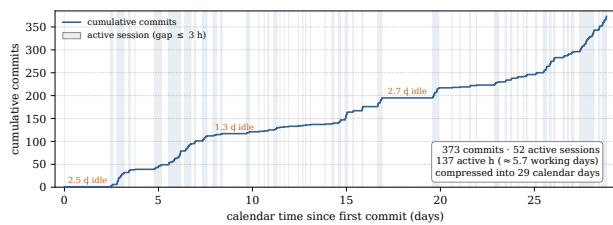


Figure 2: Implementation effort from the version-control log. Cumulative commits against calendar time; shaded bands are active sessions (consecutive commits no more than three hours apart). The long flat stretches are idle gaps. The 373 commits cluster into 52 active sessions totalling  $\sim 137$  hours of work—about 5.7 working days—spread across a 29-day calendar window.